

Modelos Avanzados de Computación

Relación 4 — Resolución comentada para estudio

Nota. Esta es la versión **comentada** de la relación. Además de las demostraciones, cada ejercicio incluye:

- cajas **Concepto** que explican desde cero las herramientas que se usan (máquinas de Turing, espacio logarítmico, certificados, reducciones, etc.), sin dar nada por sabido;
- cajas **¿Por qué este paso?** que justifican *por qué* se hace cada cosa de esa manera;
- cajas **¡Cuidado!** con los errores típicos y los matices delicados que suelen caer en examen.

Los resultados son estándar en teoría de la complejidad (Sipser, Papadimitriou, Garey–Johnson); donde un argumento sigue de cerca esos textos se indica.

Índice

Conceptos base	1
1 Grafos dirigidos acíclicos (DAG)	3
2 Grafos bipartitos	5
3 P es cerrada para unión e intersección	7
4 Cierre de clases de funciones por composición polinómica	7
5 $L = \{wcw : w \in \{0,1\}^*\}$ se reconoce en espacio $\log n$	10
6 Si $L \in P$ entonces $L^* \in P$	11
7 Si $L \in NP$ entonces $L^* \in NP$	12
8 NP es cerrada para unión e intersección	13
9 Si $NP \neq CoNP$ entonces $P \neq NP$	13
10 Paréntesis bien emparejados está en L	14
11 Dos tipos de paréntesis	14
12 El palíndromo requiere $O(n)$ sin retroceder	16
13 $NP \neq ESPACIO(n)$	17

Conceptos base (léelo antes de empezar)

Casi toda la relación gira en torno a unas pocas ideas. Las reunimos aquí para no repetirlas en cada ejercicio.

▷ Concepto: Máquina de Turing (MT) y por qué medimos “recursos”

Una **máquina de Turing** es el modelo matemático de “ordenador ideal”: una o varias cintas infinitas divididas en casillas, una cabeza que lee/escribe símbolos y se mueve, y un control finito de estados. Todo algoritmo se puede describir como una MT. La usamos porque permite *medir con precisión* dos recursos:

- **Tiempo:** número de pasos hasta parar.
- **Espacio (memoria):** número de casillas distintas que se llegan a usar.

Cuando decimos “tiempo polinómico” queremos decir: el número de pasos está acotado por n^c para alguna constante c , donde n es la longitud de la entrada. “Espacio logarítmico” significa que la memoria usada es $O(\log n)$.

▷ Concepto: Cinta de entrada de *solo lectura* (clave para el espacio)

Para medir espacios *sublineales* (menos que n) se usa un modelo con **dos cintas**:

- una **cinta de entrada de solo lectura**: contiene x , la cabeza puede moverse libremente por ella y releer, *pero no se cuenta* como memoria (es “de fuera”);
- una **cinta de trabajo** de lectura/escritura: *esta sí* se cuenta, y es donde medimos el espacio.

¿Por qué esta separación? Porque si contáramos la cinta de entrada, ningún algoritmo podría usar menos de n espacio (solo guardar la entrada ya cuesta n). Al no contarla, tiene sentido hablar de algoritmos que usan $O(\log n)$ memoria *de trabajo* aunque la entrada sea grande. Esto es lo que hace posible la clase L.

▷ Concepto: Por qué un índice cuesta $O(\log n)$ bits

Una idea que se repite muchas veces: para guardar un número (un contador, una posición, un índice) entre 0 y n , se necesitan $\lceil \log_2(n+1) \rceil$ bits en binario. Por eso “llevar unos cuantos punteros/contadores acotados por n ” ocupa $O(\log n)$ espacio. Es el truco central de los ejercicios 5, 10 y 11.

▷ Concepto: Las clases P, NP, CoNP, L, ESPACIO(n)

- P = problemas decidibles por una MT *determinista* en **tiempo** polinómico. Intuición: “resolubles de forma eficiente”.
- NP = problemas en los que, si la respuesta es “Sí”, existe una **prueba corta** (un *certificado*) que se puede **verificar** en tiempo polinómico. Intuición: “fácil de comprobar, quizá difícil de encontrar”.
- CoNP = los *complementarios* de los problemas de NP (intercambiar “Sí” y “No”). Aquí lo fácil de certificar es el “No”.
- L = ESPACIO($\log n$) = problemas decidibles en **espacio** logarítmico (de trabajo).
- ESPACIO(n) = problemas decidibles en **espacio** lineal (memoria de trabajo $O(n)$).

Relaciones básicas: $L \subseteq P \subseteq NP$ y $L \subseteq \text{ESPACIO}(n)$. Cuáles de estas inclusiones son estrictas es, en general, un problema abierto (incluido el famoso P vs NP).

▷ Concepto: Certificado y verificador (la definición operativa de NP)

Un problema P está en NP si existe un **verificador** V (algoritmo determinista polinómico) y un polinomio q tales que:

$$P(x) = \text{“Sí”} \iff \exists c \text{ con } |c| \leq q(|x|) \text{ tal que } V(x, c) = 1.$$

La cadena c es el **certificado** (o “testigo”): una pista que demuestra que la respuesta es “Sí”. La condición $|c| \leq q(|x|)$ obliga a que la pista sea *corta* (polinómica). Ejemplo: para “¿es este grafo coloreable con 3 colores?”, un certificado es la propia asignación de colores; verificarla (mirar que ninguna arista tenga dos extremos del mismo color) es trivial y rápido. Esta forma de ver NP se usa en los ejercicios 7 y 8.

▷ Concepto: Reducción en espacio logarítmico ($P_1 \propto P_2$)

Reducir P_1 a P_2 significa *transformar* cualquier entrada x de P_1 en una entrada $f(x)$ de P_2 de modo que la respuesta no cambie:

$$P_1(x) = \text{“Sí”} \iff P_2(f(x)) = \text{“Sí”}.$$

En esta asignatura se exige además que f se pueda calcular en **espacio logarítmico**. Intuición: “si sé resolver P_2 , y puedo traducir baratamente P_1 a P_2 , entonces también sé resolver P_1 ”. Es la herramienta para comparar la dificultad de problemas y la base del ejercicio 13.

▷ Concepto: Notación $O(\cdot)$ y “ $= o(\cdot)$ ”

$g(n) = O(h(n))$ significa que, a partir de cierto punto, g no crece más rápido que h salvo un factor constante (“ g está dominada por h ”). $g(n) = o(h(n))$ es más fuerte: $g/h \rightarrow 0$, es decir h crece *estrictamente* más rápido. Estas notaciones se usan en todo el ejercicio 4 y en el teorema de jerarquía del 13.

1 Grafos dirigidos acíclicos (DAG)

▷ Concepto: Vocabulario de grafos dirigidos

Un **grafo dirigido** $G = (V, E)$ tiene nodos (vértices) V y arcos *con sentido* E : un arco (u, v) va “de u hacia v ”. Un **ciclo dirigido** es un camino que respeta los sentidos y vuelve al punto de partida ($v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$). **Acíclico** = sin ciclos dirigidos (se abrevia DAG, *directed acyclic graph*). Una **fuentes** es un nodo sin arcos *entrantes* (nadie apunta a él); un **sumidero** es uno sin arcos *salientes*.

(a) Todo DAG finito tiene una fuente

Proposición 1. *Todo grafo dirigido finito y acíclico posee al menos una fuente.*

¿Por qué este paso?

Queremos una fuente porque será el “primer” nodo del orden que construiremos en (b). La estrategia es típica: suponemos que *no* existe y derivamos un absurdo (un ciclo), contradiciendo que el grafo sea acíclico.

Demostración. Razonamos por contradicción. Sea $G = (V, E)$ finito y acíclico, y supongamos que *ningún* nodo es fuente; es decir, todo nodo tiene al menos un arco entrante.

Construimos un camino hacia atrás. Tomamos un nodo cualquiera v_0 . Como v_0 no es fuente, existe un arco $(v_1, v_0) \in E$. Como v_1 tampoco es fuente, existe $(v_2, v_1) \in E$. Repitiendo, generamos una sucesión

$$\dots \rightarrow v_3 \rightarrow v_2 \rightarrow v_1 \rightarrow v_0,$$

en la que cada v_{i+1} tiene un arco hacia v_i . Como $|V| = n$ es finito, entre los $n + 1$ primeros nodos $v_0, v_1, \dots, v_n$ debe haber, por el principio del palomar, dos índices $i < j$ con $v_i = v_j$. El tramo

$$v_j \rightarrow v_{j-1} \rightarrow \dots \rightarrow v_i = v_j$$

es entonces un **ciclo dirigido**, lo que contradice que G sea acíclico. Por tanto algún nodo no tiene arco entrante: G tiene una fuente. ■

▷ Concepto: Principio del palomar

Si repartimos $n + 1$ objetos en n cajas, alguna caja recibe ≥ 2 objetos. Aquí: tenemos $n + 1$ nodos en la sucesión, pero solo n nodos distintos disponibles en el grafo, así que dos de ellos *tienen* que coincidir. Es lo que cierra el ciclo.

Nota. Por simetría (invirtiendo todos los arcos) el mismo argumento prueba que todo DAG finito tiene también un *sumidero*.

(b) Numeración topológica

Proposición 2. Un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar sus nodos $1, 2, \dots, n$ de modo que todo arco vaya de un número menor a uno mayor.

▷ Concepto: Demostrar un “si y solo si” ($\Leftrightarrow$)

Una equivalencia “ $A \Leftrightarrow B$ ” se prueba en **dos direcciones** separadas: “ $A \Rightarrow B$ ” y “ $B \Rightarrow A$ ”. Aquí A = “es acíclico” y B = “existe la numeración”. Hacemos primero la dirección fácil ($B \Rightarrow A$) y luego la constructiva ($A \Rightarrow B$).

Demostración. ($\Leftarrow$) Supongamos que existe tal numeración $\nu : V \rightarrow \{1, \dots, n\}$ con $\nu(u) < \nu(v)$ para todo arco (u, v) . Si hubiera un ciclo $w_1 \rightarrow w_2 \rightarrow \dots \rightarrow w_k \rightarrow w_1$, encadenando las desigualdades obtendríamos

$$\nu(w_1) < \nu(w_2) < \dots < \nu(w_k) < \nu(w_1),$$

es decir $\nu(w_1) < \nu(w_1)$, absurdo. Luego G es acíclico.

($\Rightarrow$) Supongamos G acíclico. Construimos la numeración por inducción sobre n .

Por el apartado (a), G tiene una fuente s . Le asignamos el número 1: $\nu(s) = 1$. Consideramos $G' = G \setminus \{s\}$ (eliminamos s y sus arcos salientes). G' sigue siendo acíclico y tiene $n - 1$ nodos; por hipótesis de inducción admite una numeración $\nu' : V \setminus \{s\} \rightarrow \{1, \dots, n - 1\}$ compatible. Definimos $\nu(v) = \nu'(v) + 1$ para $v \neq s$.

Comprobamos que todo arco va de menor a mayor:

- Arcos que salen de s : van de $\nu(s) = 1$ a algún $\nu(v) \geq 2$. Correcto.
- Arcos que no tocan s : estaban en G' y eran crecientes para ν' , luego siguen siendo crecientes para $\nu = \nu' + 1$. Correcto.
- No hay arcos *entrantes* a s porque s es fuente.

Esto completa la inducción. ■

▷ Concepto: Inducción sobre el tamaño, y qué es un “orden topológico”

La **inducción** funciona aquí porque al quitar la fuente obtenemos un grafo más pequeño *del mismo tipo* (sigue siendo DAG): resolvemos el caso pequeño y “pegamos” la fuente delante. El orden resultante se llama **orden topológico**: es exactamente “colocar los nodos en fila de forma que todas las flechas apunten hacia la derecha”. Existe *si y solo si* el grafo es acíclico, que es justo lo que dice esta proposición.

¿Por qué este paso?

¿Por qué G' sigue siendo acíclico? Porque **quitar** nodos o arcos nunca puede *crear* un ciclo (un ciclo en G' ya estaría en G). Esto es lo que nos permite aplicar la hipótesis de inducción sin problemas.

(c) Algoritmo polinómico para decidir si un grafo es acíclico

La prueba de (b) es constructiva y da directamente el algoritmo (eliminación iterativa de fuentes, conocido como algoritmo de Kahn):

Algoritmo ESACICLICO($G = (V, E)$):

1. Calcular el grado de entrada $d^-(v)$ de cada nodo. Inicializar una cola Q con todos los nodos de grado de entrada 0 (las fuentes). Contador $c \leftarrow 0$.
2. Mientras $Q \neq \emptyset$:
 - (a) Extraer u de Q ; $c \leftarrow c + 1$.
 - (b) Para cada arco (u, w) : decrementar $d^-(w)$; si queda en 0, añadir w a Q .
3. Si $c = |V|$ devolver ACICLICO; en otro caso devolver TIENE CICLO.

▷ Concepto: “Grado de entrada” y por qué lo decrementamos

El **grado de entrada** $d^-(v)$ es el número de arcos que apuntan a v . Un nodo es fuente *ahora mismo* si $d^-(v) = 0$. Cuando “eliminamos” un nodo u , sus arcos salientes desaparecen, así que cada vecino w pierde un arco entrante: por eso hacemos $d^-(w) \leftarrow d^-(w) - 1$. Si con eso w se queda a 0, w se ha convertido en una nueva fuente y entra en la cola. Así simulamos “quitar la fuente y repetir” sin tocar el grafo físicamente.

Corrección. Si G es acíclico, por (a) en cada paso del subgrafo restante siempre queda una fuente, así que el proceso elimina los n nodos y $c = n$. Recíprocamente, si el proceso se detiene con $c < n$, todos los nodos restantes tienen grado de entrada ≥ 1 , luego (por el argumento de (a) aplicado al subgrafo restante) ese subgrafo contiene un ciclo, y por tanto G también.

Complejidad. Cada arco se procesa exactamente una vez (al eliminar su origen) y cada nodo se encola/desencola una vez. El coste es $O(|V| + |E|)$, lineal y en particular polinómico.

¿Por qué este paso?

¿Por qué nos vale “polinómico” y no buscamos algo más rápido? Porque el enunciado solo pide *demostrar que el problema es tratable* (está en P). $O(|V| + |E|)$ es de hecho óptimo (hay que mirar toda la entrada), pero lo único que el ejercicio exige es la cota polinómica.

2 Grafos bipartitos**▷ Concepto: Grafo no dirigido, ciclo, longitud, bipartito**

Ahora los arcos no tienen sentido: una **arista** $\{u, v\}$ une dos vértices. La **longitud** de un ciclo es su número de aristas. Un grafo es **bipartito** si sus vértices se pueden separar en dos grupos A y B (no necesariamente del mismo tamaño) de modo que *toda* arista tenga un extremo en A y el otro en B — nunca dos del mismo grupo. Pensar en “2-coloreado”: pintar cada vértice de rojo o azul sin que dos vecinos compartan color.

(a) Bipartito $\Leftrightarrow$ todos los ciclos de longitud par

Proposición 3. Un grafo $G = (V, E)$ es bipartito si y solo si todos sus ciclos tienen longitud par.

Demostración. ($\Rightarrow$) Supongamos G bipartito con partición $V = A \dot{\cup} B$. Sea $v_0, v_1, \dots, v_k = v_0$ un ciclo. Cada arista cambia de lado: si $v_0 \in A$ entonces $v_1 \in B$, $v_2 \in A$, $\dots$; en general $v_i \in A$ si i es par y $v_i \in B$ si i es impar. Como el ciclo se cierra en $v_k = v_0 \in A$, el índice k ha de ser par. Luego todo ciclo tiene longitud par.

($\Leftarrow$) Supongamos que todos los ciclos son de longitud par. Trabajamos por componentes conexas (la unión de las biparticiones de cada componente da una bipartición del grafo entero). Fijada una componente, elegimos una raíz r y definimos, para cada vértice v , la distancia $d(v)$ = longitud del camino más corto de r a v . Coloreamos:

$$A = \{v : d(v) \text{ par}\}, \quad B = \{v : d(v) \text{ impar}\}.$$

Veamos que ninguna arista une dos vértices del mismo color. Sea $\{u, v\} \in E$. Si fuera $d(u) \equiv d(v) \pmod{2}$, consideremos los caminos más cortos P_u de r a u y P_v de r a v . Sea w su último vértice común. Los tramos $w \rightarrow u$ y $w \rightarrow v$ tienen longitudes $d(u) - d(w)$ y $d(v) - d(w)$, de la misma paridad. El recorrido

$$w \rightarrow \dots \rightarrow u - v \rightarrow \dots \rightarrow w$$

(bajar a u , usar la arista $\{u, v\}$, subir desde v) es un ciclo de longitud $(d(u) - d(w)) + 1 + (d(v) - d(w))$, que es **impar** (suma de dos números de igual paridad más 1). Esto contradice la hipótesis. Por tanto toda arista une colores distintos y la componente es bipartita. ■

▷ Concepto: “ $\pmod{2}$ ” y paridad

“ $d(u) \equiv d(v) \pmod{2}$ ” significa que $d(u)$ y $d(v)$ tienen la *misma paridad* (ambos pares o ambos impares). La diferencia de dos números de igual paridad es par; al sumarle 1 queda impar. Esa es toda la aritmética que usa la prueba.

¿Por qué este paso?

La idea profunda: en un grafo bipartito, la paridad de la distancia a la raíz determina el color, y *nunca* cambia de forma inconsistente. Un ciclo impar es justo el obstáculo que haría imposible asignar colores de forma coherente (al recorrerlo volveríamos al inicio con el color cambiado). Por eso “sin ciclos impares” $\equiv$ “coloreable con 2 colores” $\equiv$ “bipartito”.

(b) Algoritmo polinómico para comprobar bipartición**▷ Concepto: BFS, recorrido en anchura**

BFS (*breadth-first search*) explora el grafo “por capas”: primero la raíz, luego sus vecinos, luego los vecinos de estos, etc. Garantiza que cada vértice se alcanza por su camino más corto, de ahí que conozcamos la paridad de $d(v)$ de forma natural mientras recorremos. Coste $O(|V| + |E|)$.

Algoritmo ESBIPARTITO(G):

1. Marcar todos los vértices como sin color.
2. Para cada vértice r aún sin color (cabecera de una componente):
 - (a) Asignar $color(r) = 0$ y lanzar un BFS desde r .
 - (b) Al visitar una arista $\{u, v\}$: si v no tiene color, asignar $color(v) = 1 - color(u)$; si ya lo tiene y $color(v) = color(u)$, devolver NO BIPARTITO.
3. Si nunca hubo conflicto, devolver BIPARTITO con la partición dada por los colores.

Corrección. El BFS asigna a cada vértice la paridad de su distancia a la raíz, exactamente el coloreado de (a). Un conflicto ($color(u) = color(v)$ en una arista) corresponde a la existencia de un ciclo impar; su ausencia certifica una bipartición válida.

Complejidad. Recorrido BFS estándar: $O(|V| + |E|)$, polinómico.

3 P es cerrada para unión e intersección

▷ Concepto: Qué significa “cerrada para una operación”

Decir que una clase $\mathcal{C}$ es **cerrada** para una operación significa que, al aplicar esa operación a elementos de $\mathcal{C}$, el resultado *sigue* en $\mathcal{C}$. Aquí: si $L_1, L_2 \in P$, queremos que $L_1 \cup L_2$ y $L_1 \cap L_2$ también estén en P . Es como comprobar que “los pares son cerrados para la suma”: $\text{par} + \text{par} = \text{par}$.

Teorema 1. Si $L_1, L_2 \in P$, entonces $L_1 \cup L_2 \in P$ y $L_1 \cap L_2 \in P$.

Demostración. Por hipótesis existen máquinas deterministas M_1 y M_2 que deciden L_1 y L_2 en tiempo $O(n^{c_1})$ y $O(n^{c_2})$. Construimos M que, con entrada x de longitud n :

1. Simula $M_1(x)$ y guarda su respuesta $b_1 \in \{\text{Sí}, \text{No}\}$.
2. Simula $M_2(x)$ y guarda su respuesta b_2 .
3. **Unión:** acepta si $b_1 = \text{Sí}$ o $b_2 = \text{Sí}$.

Intersección: acepta si $b_1 = \text{Sí}$ y $b_2 = \text{Sí}$.

M es determinista y siempre para. Su tiempo es a lo sumo $O(n^{c_1}) + O(n^{c_2}) + O(1) = O(n^{\max(c_1, c_2)})$, polinómico. Y $L(M) = L_1 \cup L_2$ (resp. $L_1 \cap L_2$). Luego ambos están en P . ■

¿Por qué este paso?

La clave es que “correr dos algoritmos polinómicos uno detrás de otro” sigue siendo polinómico: la *suma* de dos polinomios es un polinomio. Por eso P aguanta combinaciones finitas sin salirse de la clase.

▷ Concepto: Cierre bajo complemento de P — se usará en el Ej. 9

P también es cerrada para el **complemento**: si decides L en tiempo polinómico de forma determinista, basta *intercambiar* las respuestas “Sí”/“No” para decidir $\bar{L}$ igual de rápido. Esto funciona porque la máquina es determinista y *siempre para* con una respuesta clara. Para NP *no* se sabe hacer esto (de ahí la pregunta NP vs $CoNP$).

4 Cierre de clases de funciones por composición polinómica

▷ Concepto: De qué va realmente este ejercicio

Tenemos *familias* de funciones (por ejemplo “todos los n^k ”). Nos preguntamos: si compongo una función de la familia con un polinomio, ¿el resultado sigue “cabiendo” (en orden de magnitud) dentro de la misma familia? Hay dos formas de componer:

- **Por la izquierda:** el polinomio va *fuera*: $p(f(n))$. Como basta tomar $p(n) = n^l$, esto es $f(n)^l$ — “elevant f a una potencia”.
- **Por la derecha:** el polinomio va *dentro*: $f(p(n)) = f(n^l)$ — “meter n^l como argumento de f ”.

Cerrada = el resultado está acotado ($= O(\cdot)$) por *alguna* función g de la misma familia.

▷ **Concepto: Por qué basta usar $p(n) = n^l$**

El enunciado lo aclara: como hablamos de *órdenes* de complejidad, cualquier polinomio $p(n) = a_k n^k + \dots$ cumple $p(n) = O(n^k)$, es decir está dominado por su término de mayor grado. Así que el “peor” polinomio, a efectos de orden, es el monómio n^l . Por eso solo necesitamos analizar $f(n)^l$ y $f(n^l)$.

A continuación analizamos las seis familias. Resumimos al final en una tabla.

(a) $\mathcal{C} = \{n^k : k > 0\}$ (**potencias de n**)

Sea $f(n) = n^k \in \mathcal{C}$.

Izquierda. $p(f(n)) = (n^k)^l = n^{kl}$, y como $kl > 0$ se tiene $n^{kl} \in \mathcal{C}$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = (n^l)^k = n^{lk} \in \mathcal{C}$. **Cerrada por la derecha.** (a) cerrada por ambos lados.

¿Por qué este paso?

Funciona porque la familia está parametrizada por el *exponente*, y al elevar/sustituir solo **multiplicamos exponentes** ($k \cdot l$), que sigue siendo un exponente positivo válido. La familia es “inmune” a estas operaciones.

(b) $\mathcal{C} = \{n \cdot k : k > 0\}$ (**funciones lineales**)

¡Cuidado!

Aquí k es un **factor** (la pendiente), no un exponente: $f(n) = kn$ es una recta. No confundir con el caso (a). Todas las funciones de esta familia son *lineales*.

Izquierda. $p(f(n)) = (kn)^l = k^l n^l$. Para $l \geq 2$ esto es $\Theta(n^l)$, un crecimiento *superlineal* que no puede acotarse por ninguna función lineal $g(n) = k'n$ (pues $n^l/n = n^{l-1} \rightarrow \infty$). **No cerrada por la izquierda.**

Derecha. $f(p(n)) = kn^l$. Igual: para $l \geq 2$ es $\Theta(n^l)$, no acotable por una recta. **No cerrada por la derecha.** (b) no cerrada por ningún lado.

¿Por qué este paso?

Las rectas son “demasiado rígidas”: solo tienen libertad en la pendiente, no en el exponente, que está fijo en 1. En cuanto el polinomio introduce un exponente $l \geq 2$, salimos de las rectas y caemos en cuadráticas/cúbicas, que ninguna recta puede dominar.

(c) $\mathcal{C} = \{k^n : k > 0\}$ (**exponenciales de base constante**)

Sea $f(n) = k^n$ (interesante para $k \geq 2$).

Izquierda. $p(f(n)) = (k^n)^l = k^{ln} = (k^l)^n$. Con $k' = k^l$ obtenemos $(k')^n \in \mathcal{C}$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = k^{n^l}$. Para $l \geq 2$ crece como k^{n^2} , muchísimo más rápido que cualquier $(k')^n$:

$$\frac{k^{n^l}}{(k')^n} = k^{n^l - n \log_k k'} \rightarrow \infty.$$

No cerrada por la derecha. (c) cerrada por la izquierda, no por la derecha.

¿Por qué este paso?

Por la izquierda solo cambiamos la *base* ($k \rightarrow k^l$): seguimos en una exponencial “simple”. Por la derecha cambiamos el *exponente* de n a n^l : pasamos de crecimiento k^n a k^{n^2} , que es una bestia muchísimo mayor que ninguna k'^n alcanza. La familia no es lo bastante amplia para contenerla.

(d) $\mathcal{C} = \{2^{n^k} : k > 0\}$ (exponencial con exponente polinómico)

Sea $f(n) = 2^{n^k}$.

Izquierda. $p(f(n)) = (2^{n^k})^l = 2^{ln^k}$. Como l es constante, para n grande $ln^k \leq n^{k+1}$, luego $2^{ln^k} \leq 2^{n^{k+1}} = O(2^{n^{k+1}})$ y $2^{ln^k} \in \mathcal{C}$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = 2^{(n^l)^k} = 2^{n^{lk}} \in \mathcal{C}$ (pues $lk > 0$). **Cerrada por la derecha.** (d) cerrada por ambos lados.

¿Por qué este paso?

Esta familia es “ancha” en el exponente (cualquier n^k vale ahí arriba), así que tanto multiplicar el exponente por una constante (ln^k , que absorbemos subiendo a n^{k+1}) como elevarlo (n^{lk}) nos deja *dentro*. Compara con (c): la diferencia es que aquí el exponente ya es *polinómico en n* , no constante.

(e) $\mathcal{C} = \{\log^k(n) : k > 0\}$ (potencias de logaritmo)

Sea $f(n) = (\log n)^k = \log^k n$ (el redondeo a entero no afecta al orden).

Izquierda. $p(f(n)) = (\log^k n)^l = \log^{kl} n \in \mathcal{C}$, pues $kl > 0$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = \log^k(n^l) = (l \log n)^k = l^k \log^k n = O(\log^k n)$, y $\log^k n \in \mathcal{C}$. **Cerrada por la derecha.** (e) cerrada por ambos lados.

▷ Concepto: La identidad clave: $\log(n^l) = l \log n$

Una propiedad básica del logaritmo: el exponente sale como factor, $\log(n^l) = l \log n$. Por eso “meter n^l dentro del log” solo añade un factor constante l (que el $O(\cdot)$ se traga), y por eso (e) y (f) son cerradas *por la derecha*. Es el mismo mecanismo que hace los logaritmos tan robustos.

(f) $\mathcal{C} = \{\log n\}$ (un único elemento)

¡Cuidado!

Esta familia tiene **una sola** función, $f(n) = \log n$. Por tanto la única g candidata para dominar el resultado es la propia $\log n$. Esto la hace mucho más frágil que (e): no podemos “subir el índice k ” porque no hay más elementos.

Izquierda. $p(f(n)) = (\log n)^l = \log^l n$. Para $l \geq 2$, $\log^l n / \log n = \log^{l-1} n \rightarrow \infty$, así que $\log^l n \neq O(\log n)$: la única g disponible no lo acota. **No cerrada por la izquierda.**

Derecha. $f(p(n)) = \log(n^l) = l \log n = O(\log n)$, dominado por la única función de la clase.

Cerrada por la derecha. (f) cerrada por la derecha, no por la izquierda.

¿Por qué este paso?

Compara (e) y (f): por la izquierda obtenemos $\log^l n$ en ambos casos, pero en (e) *esa* función pertenece a la familia (hay un $\log^k$ para cada k), mientras que en (f) no existe nadie más que $\log n$ para acotarla. La moraleja: el cierre no depende solo del resultado de la operación, sino de *qué funciones tiene la familia disponibles* para dominarlo.

Resumen

Clase	Izquierda $f(n)^l$	Derecha $f(n^l)$
(a) $\{n^k\}$	Sí	Sí
(b) $\{k n\}$	No	No
(c) $\{k^n\}$	Sí	No
(d) $\{2^{n^k}\}$	Sí	Sí
(e) $\{\log^k n\}$	Sí	Sí
(f) $\{\log n\}$	No	Sí

Nota. Patrón conceptual: la composición *por la izquierda* eleva f a una potencia; sobrevive cuando la familia es estable al elevar a potencias constantes (familias paramétricas en el exponente o el índice del logaritmo). La composición *por la derecha* mete n^l dentro de f ; sobrevive cuando “elevar el argumento a una potencia” equivale a moverse dentro de la familia. Las familias con un solo grado de libertad mal situado (lineales, o el logaritmo aislado) fallan en el lado que ataca ese grado de libertad.

5 $L = \{wcw : w \in \{0, 1\}^*\}$ se reconoce en espacio $\log n$

▷ Concepto: Qué pide el problema

$L = \{wcw\}$ son las cadenas formadas por una palabra w de ceros y unos, una marca c en medio, y la misma palabra w otra vez. Ejemplos: 0c0, 101c101 son de L ; 10c01 no (las mitades difieren). Queremos decidir la pertenencia usando solo $O(\log n)$ memoria de trabajo.

¡Cuidado!

La tentación es *copiar* la primera mitad w a la cinta de trabajo y luego compararla. Pero copiar w cuesta $|w| \approx n/2$ espacio: **lineal**, no logarítmico. Hay que evitarlo a toda costa.

Teorema 2. $L = \{wcv : w \in \{0,1\}^*\}$ pertenece a $L = \text{ESPACIO}(\log n)$.

Idea. En vez de copiar, comparamos las dos mitades *posición a posición* usando **punteros** (índices). Comparar el carácter i de la izquierda con el carácter i de la derecha solo requiere recordar el número i , que cabe en $O(\log n)$ bits. Aquí es esencial que la cinta de entrada sea de solo lectura y podamos movernos por ella libremente (concepto base).

Demostración. Modelo: cinta de entrada de solo lectura + cinta de trabajo (la que se mide). Entrada de longitud n .

Fase 1 (formato). Recorremos la entrada contando cuántas c hay y anotando la posición m de la (única) c . El contador y m son $\leq n$, luego $O(\log n)$ bits. Si el número de c no es exactamente 1, rechazar. Tras esto, la izquierda es $u = x_1 \cdots x_{m-1}$ y la derecha $v = x_{m+1} \cdots x_n$.

Fase 2 (longitudes). Para que sea wcv debe cumplirse $|u| = |v|$, o sea $m - 1 = n - m$. Se comprueba con aritmética sobre m y n , en $O(\log n)$. Si no, rechazar.

Fase 3 (comparación). Para $i = 1, 2, \dots, m - 1$:

- mover la cabeza de lectura a la posición i y leer x_i (un bit);
- mover la cabeza a la posición $m + i$ y leer x_{m+i} ;
- si $x_i \neq x_{m+i}$, rechazar.

Si todas coinciden, aceptar.

Espacio. En la cinta de trabajo solo hay, en cada momento, un contador (el índice i , o m , o el conteo de c) y uno o dos bits leídos. Todo índice es $\leq n$, luego $O(\log n)$ bits; el número de variables es constante. Total: $O(\log n)$. ■

¿Por qué este paso?

¿Por qué “ $O(\log n)$ ” y no contar el tiempo? El tiempo es $O(n^2)$ (por los reposicionamientos de la cabeza), pero el ejercicio pide *espacio*, y en espacio ganamos muchísimo: pasamos de memorizar media palabra ($n/2$ bits) a memorizar un índice ($\log n$ bits). El truco “releer en lugar de almacenar” reaparecerá en los ejercicios 10–12.

6 Si $L \in P$ entonces $L^* \in P$

▷ Concepto: La estrella de Kleene L^*

L^* es el conjunto de cadenas que se obtienen *concatenando cero o más* trozos de L . Es decir, $x \in L^*$ si x se puede partir como $x = w_1 w_2 \cdots w_k$ con cada $w_j \in L$ (y la cadena vacía ε siempre está, con $k = 0$). Ejemplo: si $L = \{ab, c\}$, entonces $abcab \in L^*$ (partido como $ab \cdot c \cdot ab$).

Teorema 3. Si $L \in P$, entonces $L^* \in P$.

Idea. El problema es “¿existe *alguna* forma de cortar x en trozos que estén todos en L ?”. Probar todas las formas de cortar sería exponencial. Usamos **programación dinámica**: resolvemos el problema para los prefijos cada vez más largos, reutilizando lo ya calculado.

▷ Concepto: Programación dinámica, en una frase

Técnica que rompe un problema en subproblemas que se *solapan*, los resuelve una sola vez y guarda las respuestas en una tabla para no repetir trabajo. Aquí el subproblema es: “¿el prefijo $x[1..j]$ pertenece a L^* ?”.

Demostración. Sea M una máquina que decide L en tiempo $O(n^c)$. Escribimos $x[i..j] = x_i \cdots x_j$ (subcadena) y por convenio $x[i..i-1] = \varepsilon$.

Definimos el vector booleano $A[0..n]$ con

$$A[j] = \text{Verdadero} \iff x[1..j] \in L^*.$$

Recurrencia (mirando dónde empieza el *último* trozo):

$$A[0] = \text{Verdadero}, \quad A[j] = \bigvee_{0 \leq i < j} \left(A[i] \wedge (x[i+1..j] \in L) \right).$$

Algoritmo. Calcular $A[0], A[1], \dots, A[n]$ en orden creciente; para cada par (i, j) se invoca M sobre $x[i+1..j]$. Aceptar si $A[n]$ es Verdadero.

Corrección. Por inducción sobre j : $A[j]$ es Verdadero exactamente cuando $x[1..j]$ se puede descomponer en factores de L , que es la definición de pertenecer a L^* . El caso base $A[0]$ es $\varepsilon \in L^*$.

Complejidad. Hay $O(n^2)$ pares (i, j) ; cada uno cuesta una llamada a M , $O(n^c)$. Total $O(n^{c+2})$, polinómico. Luego $L^* \in P$. ■

¿Por qué este paso?

¿Por qué mirar “dónde empieza el último trozo”? Porque si conocemos ese punto de corte i , el problema se parte limpiamente en dos piezas *independientes*: “¿ $x[1..i]$ ya estaba bien partido?” (eso es $A[i]$, ya calculado) y “¿el resto $x[i+1..j]$ es un único trozo de L ?” (una llamada a M). El “o” ($\vee$) recorre todos los posibles puntos de corte: con que *uno* funcione, basta.

7 Si $L \in NP$ entonces $L^* \in NP$

Teorema 4. Si $L \in NP$, entonces $L^* \in NP$.

Idea. Usamos la caracterización por *certificado* (concepto base). Para convencer a alguien de que $x \in L^*$ basta darle: (1) dónde están los cortes que parten x en trozos, y (2) para cada trozo, el certificado de que ese trozo está en L . Todo eso es corto y se comprueba rápido.

Demostración (versión verificador). Como $L \in NP$, hay un verificador V y un polinomio q con $w \in L \iff \exists c_w (|c_w| \leq q(|w|)) : V(w, c_w) = 1$, y V corre en tiempo polinómico.

Definimos un verificador V^* para L^* . Dado x ($|x| = n$), el certificado propuesto es

$$(0 = i_0 < i_1 < \dots < i_k = n, \quad c_1, \dots, c_k),$$

donde los i_j marcan los cortes (el trozo j es $w_j = x[i_{j-1}+1..i_j]$) y c_j certifica $w_j \in L$. Si $x = \varepsilon$ se acepta ($k = 0$). V^* :

1. comprueba que $0 = i_0 < i_1 < \dots < i_k = n$ (cortes válidos que cubren todo x);
2. para cada j , extrae w_j y verifica $V(w_j, c_j) = 1$;
3. acepta si todo va bien.

Corrección. $x \in L^*$ si y sólo si existe una partición en trozos de L , lo cual equivale a la existencia de cortes válidos y certificados aceptados.

Tamaño del certificado. A lo sumo $k \leq n$ cortes, cada uno un índice $\leq n$ ($O(n \log n)$ bits); cada c_j mide $\leq q(n)$ y, como $\sum_j |w_j| = n$, la suma total de certificados es $\leq n \cdot q(n)$: polinómica.

Tiempo. Comprobar cortes es lineal; ejecutar V en cada trozo es polinómico y se hace $\leq n$ veces. Polinómico en total. Luego $L^* \in NP$. ■

▷ Concepto: La diferencia entre el Ej. 6 y el Ej. 7

Son el “mismo” problema (L^*) pero con distinta hipótesis sobre L :

- Ej. 6 ($L \in P$): podemos *comprobar* “ $x[i+1..j] \in L$ ” directamente, así que basta la programación dinámica determinista.
- Ej. 7 ($L \in NP$): ya no podemos comprobarlo de golpe, pero *sí* verificar un certificado. Así que el certificado global de L^* = los cortes + un certificado por trozo.

De hecho se podría hacer el 7 con la misma programación dinámica del 6, sustituyendo cada test “ $\in L$ ” por “adivinar y verificar el certificado del trozo”. Ambos enfoques son válidos.

8 NP es cerrada para unión e intersección

Teorema 5. Si $L_1, L_2 \in \text{NP}$, entonces $L_1 \cup L_2 \in \text{NP}$ y $L_1 \cap L_2 \in \text{NP}$.

Idea. Como en el Ej. 3, pero ahora con certificados en lugar de máquinas deterministas. La pregunta clave en cada caso es: “¿qué pista (certificado) basta para convencer de que x está en la unión / intersección?”.

Demostración. Sean V_1, V_2 verificadores polinómicos y q_1, q_2 polinomios con $x \in L_i \iff \exists c_i (|c_i| \leq q_i(|x|)) : V_i(x, c_i) = 1$.

Unión. El certificado es un par (etiqueta, testigo): un bit $b \in \{1, 2\}$ que dice “a cuál de los dos pertenece x ” más el certificado c_b correspondiente. El verificador $V_{\cup}(x, (b, c))$ ejecuta $V_b(x, c)$. Entonces

$$x \in L_1 \cup L_2 \iff (\exists c_1 : V_1(x, c_1) = 1) \text{ o } (\exists c_2 : V_2(x, c_2) = 1) \iff \exists (b, c) : V_{\cup}(x, (b, c)) = 1.$$

Intersección. El certificado es el par (c_1, c_2) de ambos testigos. $V_{\cap}(x, (c_1, c_2))$ ejecuta $V_1(x, c_1)$ y $V_2(x, c_2)$ y acepta si ambos aceptan. Entonces

$$x \in L_1 \cap L_2 \iff (\exists c_1 : V_1(x, c_1) = 1) \text{ y } (\exists c_2 : V_2(x, c_2) = 1) \iff \exists (c_1, c_2) : V_{\cap}(x, (c_1, c_2)) = 1.$$

En ambos casos el certificado es de tamaño polinómico y el verificador corre en tiempo polinómico. Luego $L_1 \cup L_2, L_1 \cap L_2 \in \text{NP}$. ■

¡Cuidado!

En la intersección, el paso “ $\exists c_1 \exists c_2 \equiv \exists (c_1, c_2)$ ” es lícito porque los dos testigos son **independientes**: elegir c_1 no condiciona la búsqueda de c_2 . Esto NO funcionaría si la operación requiriese cuantificadores alternados ($\exists \dots \forall \dots$); ahí es justo donde NP deja de ser suficiente y aparecen clases superiores. Pero unión e intersección solo usan “ $\exists$ ”, así que NP aguanta.

9 Si $\text{NP} \neq \text{CoNP}$ entonces $\text{P} \neq \text{NP}$

▷ Concepto: Demostrar por el contrarrecíproco

Probar “ $A \Rightarrow B$ ” es *exactamente* lo mismo que probar “no $B \Rightarrow$ no A ” (el **contrarrecíproco**). A veces una dirección es mucho más fácil. Aquí, en vez de “ $\text{NP} \neq \text{CoNP} \Rightarrow \text{P} \neq \text{NP}$ ”, probamos lo equivalente y más manejable: “ $\text{P} = \text{NP} \Rightarrow \text{NP} = \text{CoNP}$ ”.

Teorema 6. Si $\text{NP} \neq \text{CoNP}$, entonces $\text{P} \neq \text{NP}$.

Demostración. Probamos el contrarrecíproco: $\text{P} = \text{NP} \Rightarrow \text{NP} = \text{CoNP}$.

Supongamos $\text{P} = \text{NP}$. Recordemos (concepto del Ej. 3) que P es cerrada para complemento: si $A \in \text{P}$, intercambiando aceptación/rechazo decidimos $\bar{A}$ igual de rápido, luego $\bar{A} \in \text{P}$.

Veamos la doble inclusión $\text{NP} = \text{CoNP}$.

$\text{NP} \subseteq \text{CoNP}$: Sea $A \in \text{NP}$. Por hipótesis $A \in \text{P}$, y como P es cerrada para complemento, $\bar{A} \in \text{P} = \text{NP}$. Pero “ $\bar{A} \in \text{NP}$ ” es justo la definición de “ $A \in \text{CoNP}$ ”. Luego $A \in \text{CoNP}$.

$\text{CoNP} \subseteq \text{NP}$: Sea $A \in \text{CoNP}$, es decir $\bar{A} \in \text{NP}$. Por hipótesis $\bar{A} \in \text{P}$, y por cierre bajo complemento $A \in \text{P} = \text{NP}$. Luego $A \in \text{NP}$.

De ambas, $\text{NP} = \text{CoNP}$. Tomando contrarrecíproco: $\text{NP} \neq \text{CoNP} \Rightarrow \text{P} \neq \text{NP}$. ■

¿Por qué este paso?

El *corazón* del argumento es la asimetría: P **sí** es cerrada para complemento (porque sus máquinas son deterministas y siempre paran), pero de NP *no* se sabe. Si P y NP fueran iguales, NP heredaría esa propiedad de P y colapsaría con su complementaria CoNP. Que la mayoría de expertos crea $NP \neq CoNP$ es, por esta vía, otra razón para creer $P \neq NP$.

▷ Concepto: Definición de CoNP, recordatorio aplicado

$A \in CoNP$ significa, por definición, que su complementario $\bar{A}$ está en NP. Por eso en la prueba “traducimos” constantemente entre “ $A \in CoNP$ ” y “ $\bar{A} \in NP$ ”: son la misma afirmación. Tenerlo claro es lo que hace la prueba casi mecánica.

10 Paréntesis bien emparejados está en L

Teorema 7. *El lenguaje de cadenas de paréntesis $\{ (,) \}$ correctamente emparejados y anidados pertenece a $L = ESPACIO(\log n)$.*

Idea. Con **un solo** tipo de paréntesis no hace falta una pila: basta un **contador** de profundidad (cuántos paréntesis hay abiertos en este momento). La regla de buena formación es local: “el contador nunca baja de cero y acaba en cero”.

Demostración. Entrada $x = x_1 \cdots x_n$ sobre $\{ (,) \}$. Mantenemos un único contador b (“balance”), $b \leftarrow 0$. Recorremos de izquierda a derecha:

- si $x_i = “(”$, hacer $b \leftarrow b + 1$;
- si $x_i = “)”$, hacer $b \leftarrow b - 1$;
- si en algún momento $b < 0$, **rechazar**.

Al terminar, **aceptar** si $b = 0$.

Corrección. Una cadena de un solo tipo está bien formada si y solo si en todo prefijo hay tantos o más “(” que “)” (nunca se cierra de más) y al final hay igual número (todo lo abierto se cierra). El contador mide exactamente $\#(- \#)$ del prefijo leído: “ $b \geq 0$ siempre” y “ $b = 0$ al final” son justo esas dos condiciones. Así, $((()())$ y $((()))$ se aceptan; $()()$ (balance -1 al primer símbolo) y $((()))()$ (balance llega a -1) se rechazan.

Espacio. El contador cumple $0 \leq b \leq n$, luego cabe en $\lceil \log_2(n+1) \rceil = O(\log n)$ bits. No se usa nada más. Está en L. ■

¿Por qué este paso?

La razón por la que un contador basta es que, con un solo tipo, *no importa cuáles* paréntesis se emparejan, solo *cuántos* hay abiertos. Toda la información de una pila de altura h se resume en el número h . Y un número $\leq n$ cuesta solo $\log n$ bits. En cuanto haya *dos* tipos (Ej. 11) esto se rompe: ya importará el orden.

11 Dos tipos de paréntesis

Teorema 8. *El lenguaje de cadenas bien formadas con dos tipos de paréntesis, $(,)$ y $[,]$, también pertenece a L.*

▷ Concepto: Por qué dos tipos complican las cosas: la pila

Con dos tipos importa el *orden* de cierre: $([]())$ es válida pero $([])$ no (se cierra un “)” cuando lo último abierto era un “[”). La estructura natural para llevar esto es una **pila**: cada apertura se apila, cada cierre debe casar con el último apilado (el del tope). Pero una pila puede crecer hasta $\Theta(n)$ símbolos: sería espacio *lineal*, no logarítmico.

Idea. El truco para mantenernos en $O(\log n)$ es **no almacenar la pila**: para cada cierre, *recalculamos* cuál es su apertura asociada recorriendo de nuevo la entrada con contadores. Pagamos más tiempo a cambio de casi nada de espacio. (Aprovechamos que la cinta de entrada es de solo lectura y la podemos releer.)

Demostración. **Validez:** la cadena es correcta si (1) está balanceada *ignorando el tipo* (contador global nunca negativo, cero al final — como en el Ej. 10), y (2) cada cierre casa en *tipo* con la apertura que lo empareja en el anidamiento.

Emparejar sin pila. Para un cierre en posición j , su apertura asociada es la posición $i < j$ tal que el segmento intermedio $x[i + 1..j - 1]$ está balanceado. Operativamente, en $O(\log n)$ espacio: recorremos x con el contador de balance b (verificando de paso la condición 1). Cada vez que hallamos un cierre en posición j , lanzamos un segundo recorrido desde el principio para localizar esa apertura i (la última posición $< j$ desde la que el sub-balance hasta $j - 1$ es 0), y comprobamos que x_i y x_j son del mismo tipo. Si no, rechazar.

Todo el almacenamiento son unos pocos contadores acotados por n (j , la candidata i , los balances), cada uno $O(\log n)$ bits; no guardamos la pila completa, la recomputamos. Espacio total: $O(\log n)$. (Tiempo: $O(n^2)$.) Luego está en L.

Segunda parte: leyendo de izquierda a derecha SIN volver atrás, requieren $O(n)$ de memoria. ■

▷ **Concepto: Modelo “online”: una pasada, sin retroceso**

Ahora cambiamos las reglas: la entrada se lee **una sola vez**, de izquierda a derecha, sin poder releer (como un flujo que pasa y no vuelve). En este modelo ya *no* podemos “recomputar la pila” como antes (eso requería releer). La pregunta es cuánta memoria necesitamos *obligatoriamente*.

▷ **Concepto: Argumento de distinguibilidad (“fooling set”)**

Técnica para demostrar *cotas inferiores* de memoria. Idea: si encontramos muchos prefijos distintos que la máquina **tiene** que tratar de forma diferente (porque su continuación correcta difiere), entonces tras leerlos debe estar en *configuraciones de memoria distintas*. Si hay N prefijos distinguibles, hacen falta $\geq \log_2 N$ bits de memoria. Es el mismo principio que dice “para distinguir N cosas necesitas $\log_2 N$ bits”.

Demostración de la cota $\Omega(n)$. Consideremos los 2^m prefijos de m aperturas, cada una de tipo redondo o cuadrado: $u_S = s_1 \cdots s_m$ con $s_t \in \{ (, [\}$. Para dos prefijos distintos $u_S \neq u_{S'}$, sea t la primera posición donde difieren los tipos. La continuación que cierra *en el orden correcto* para u_S —el “espejo” $\bar{s}_m \cdots \bar{s}_1$, donde $\bar{(} =)$ y $\bar{[} =]$ — completa u_S a una cadena **válida**, pero aplicada a $u_{S'}$ produce un **desajuste de tipo**, dando cadena **inválida**.

Por tanto la máquina debe quedar en estados de memoria *distintos* tras leer u_S y $u_{S'}$: si coincidieran, al recibir la misma continuación se comportaría igual con ambas (sin poder releer el prefijo), aceptando o rechazando las dos a la vez, contradicción.

Así hay $\geq 2^m$ configuraciones de memoria distinguibles tras leer m símbolos, lo que exige $\geq \log_2(2^m) = m$ bits de memoria. Como m puede ser $\Theta(n)$, la memoria necesaria es $\Omega(n)$ (lineal). ■

¿Por qué este paso?

¿Por qué usamos el “espejo” como continuación? Porque es la única forma de cerrar bien un anidamiento: lo último que se abre es lo primero que se cierra. Al fijar esa continuación para u_S , forzamos que cualquier $u_{S'}$ con tipos distintos falle. Así fabricamos a propósito un montón de pares que la máquina no puede confundir, y de ahí sale la cota.

¡Cuidado!

No te confundas: la primera parte (en L, $O(\log n)$) y la segunda (requiere $O(n)$) *no* se contradicen. Hablan de **modelos distintos**: la primera *permite releer* la entrada (acceso de solo lectura con reposicionamiento), la segunda *no*. La cota logarítmica se consigue precisamente *gracias* a poder releer y recomputar la pila; si lo prohibimos, hay que memorizar la secuencia de tipos, que son $\Theta(n)$ bits.

12 El palíndromo requiere $O(n)$ sin retroceder

▷ Concepto: Palíndromo

Una cadena w es **palíndromo** si se lee igual al derecho y al revés: $w = w^R$ (donde w^R es w invertida). Ejemplos: 0110, 10101. El problema es decidir si la entrada es un palíndromo, en el modelo de una sola pasada sin retroceso.

Teorema 9. Reconocer palíndromos requiere memoria $\Omega(n)$ (lineal) si la entrada se lee de izquierda a derecha sin poder volver atrás.

Idea. Mismo argumento de distinguibilidad del Ej. 11. Exhibimos exponencialmente muchos prefijos que la máquina debe distinguir (su continuación correcta difiere), forzando 2^m configuraciones de memoria y por tanto $\Omega(m) = \Omega(n)$ bits.

Demostración. Sea $L_{\text{pal}} = \{w : w = w^R\}$ sobre $\{0, 1\}$, y M una máquina que lee la entrada una vez, de izquierda a derecha, con memoria de trabajo $s(n)$.

Fijado m , tomamos los 2^m prefijos $u \in \{0, 1\}^m$. Afirmamos que M debe llegar a configuraciones de memoria distintas tras leer dos prefijos distintos $u \neq u'$.

En efecto, supongamos lo contrario: M alcanza la *misma* configuración tras leer u y tras u' , con $u \neq u'$ (difieren en alguna posición t). Usamos la continuación $z = u^R$ (el reverso de u). Entonces:

- $uz = uu^R$ es palíndromo (forma ww^R): M debe **aceptar**.
- $u'z = u'u^R$ **no** es palíndromo: en la posición t (primera mitad) vale u'_t , mientras que su simétrica (segunda mitad) vale $u_t \neq u'_t$: M debe **rechazar**.

Pero las configuraciones tras u y u' coinciden por hipótesis, y como M no puede volver atrás, su comportamiento al leer z es idéntico en ambos casos: aceptaría las dos o rechazaría las dos. Contradicción.

Luego las 2^m configuraciones son distintas. El número de configuraciones con s celdas (control de Q estados, cabeza en s posiciones, alfabeto de trabajo de tamaño γ) es $\leq Q \cdot s \cdot \gamma^s = 2^{O(s)}$. Necesitamos $2^{O(s)} \geq 2^m$, de donde $s = \Omega(m)$. Tomando $m = \lfloor n/2 \rfloor$, $s(n) = \Omega(n)$. ■

▷ Concepto: Por qué “#configuraciones = $2^{O(s)}$ ”

Una “configuración de memoria” es una foto del estado interno: qué estado de control, dónde está la cabeza, y qué contiene la cinta de trabajo. Con s celdas y un alfabeto de γ símbolos, los contenidos posibles son γ^s ; multiplicado por Q estados y s posiciones de cabeza da $Q s \gamma^s$, que es $2^{O(s)}$. La moraleja: **con s bits de memoria solo puedes “recordar” $2^{O(s)}$ situaciones distintas**. Si necesitas distinguir 2^m , te hacen falta $s \geq \Omega(m)$ bits.

¡Cuidado!

Compara con el Ej. 5: $\{wcw\}$ sí está en L, y el palíndromo $\{ww^R\}$ también lo estaría *con relectura* (comparando x_i con x_{n+1-i} por punteros). La cota $\Omega(n)$ de este ejercicio depende **críticamente** de “sin volver atrás”: es lo que prohíbe los punteros y obliga a memorizar media cadena entera. Si en un examen ves “espacio $\log n$ ” y “ $\Omega(n)$ ” para problemas parecidos, casi

siempre la diferencia es el modelo de acceso a la entrada.

13 $NP \neq ESPACIO(n)$

¡Cuidado!

Este ejercicio es *el más difícil* de la relación. Un punto clave para no perderse: **no** vamos a decir cuál clase contiene a cuál (de hecho se desconoce si $NP \subseteq ESPACIO(n)$). Solo demostraremos que *no pueden ser la misma*. La táctica: encontrar una **propiedad** que una clase tiene y la otra no. Dos clases iguales tendrían las mismas propiedades; si difieren en una, son distintas.

Idea. La propiedad elegida (sugerida por el enunciado) es “ser cerrada para reducciones en espacio logarítmico”. Mostraremos: (**Paso 1**) NP sí lo es; (**Paso 2**) $ESPACIO(n)$ no lo es. Conclusión inmediata: $NP \neq ESPACIO(n)$.

Definición 1 (cierre para reducciones en L). Una clase $\mathcal{D}$ es *cerrada para reducciones en espacio logarítmico* si: siempre que $P_1 \propto P_2$ (reducción calculable en espacio log) y $P_2 \in \mathcal{D}$, entonces también $P_1 \in \mathcal{D}$.

▷ Concepto: Qué dice esta propiedad, en cristiano

“ $\mathcal{D}$ es cerrada para log-reducciones” significa: *si un problema se reduce baratamente a otro que está en $\mathcal{D}$, el primero también está en $\mathcal{D}$* . Es una forma de decir que $\mathcal{D}$ “respeta” la relación de dificultad: nada que sea “más fácil o igual” que un miembro de $\mathcal{D}$ se queda fuera.

Paso 1: NP es cerrada para reducciones en L

Lema 1. Si $P_1 \propto P_2$ y $P_2 \in NP$, entonces $P_1 \in NP$.

Demostración. Sea f la función de reducción, calculable en espacio $O(\log n)$ y por tanto en tiempo polinómico (ver concepto siguiente); en particular $|f(x)|$ es polinómico. Cumple $x \in P_1 \iff f(x) \in P_2$. Como $P_2 \in NP$, sea V_2 su verificador polinómico.

Verificador para P_1 : dado x y un certificado c , (1) calcular $y = f(x)$, (2) aceptar si $V_2(y, c) = 1$. Entonces

$$x \in P_1 \iff f(x) \in P_2 \iff \exists c (|c| \leq q(|f(x)|)) : V_2(f(x), c) = 1,$$

y como $|f(x)|$ es polinómico en $|x|$, el certificado es corto y la verificación polinómica. Luego $P_1 \in NP$. ■

▷ Concepto: Por qué “espacio log” implica “tiempo polinómico”

Una máquina que usa $O(\log n)$ espacio de trabajo tiene un número *limitado* de configuraciones distintas: a lo sumo $2^{O(\log n)} = n^{O(1)}$ (polinómicas). Si una máquina que siempre para no repitiera configuración (si lo hiciera, entraría en bucle infinito), entonces da a lo sumo $n^{O(1)}$ pasos. Conclusión: $L \subseteq P$ y, en general, todo cómputo log-espacial corre en tiempo polinómico. Por eso la reducción, aunque definida por espacio, produce una salida $f(x)$ de tamaño solo polinómico.

Paso 2: ESPACIO(n) *no* es cerrada para reducciones en L▷ **Concepto:** Diagonalización y el Teorema de Jerarquía de Espacio

Diagonalización es la técnica (la de Cantor, la del problema de la parada) que construye un objeto que difiere de cada elemento de una lista en al menos un punto, garantizando que “no está en la lista”. Aplicada a complejidad, permite construir un problema que ninguna máquina con *pocos* recursos puede resolver, pero sí una con *algo más* de recursos. El resultado:

Teorema 10 (Jerarquía de espacio; Stearns–Hartmanis–Lewis). Si $s_1(n) = o(s_2(n))$ (y s_2 es “construible en espacio”, $s_2(n) \geq \log n$), entonces la inclusión $\text{ESPACIO}(s_1(n)) \subsetneq \text{ESPACIO}(s_2(n))$ es **estricta**. En particular, como $n = o(n^2)$:

$$\text{ESPACIO}(n) \subsetneq \text{ESPACIO}(n^2).$$

¿Por qué este paso?

Necesitamos este teorema porque nos da, “gratis”, un problema concreto que está en $\text{ESPACIO}(n^2)$ pero *no* en $\text{ESPACIO}(n)$: justo la pieza que separa una clase de la otra. Sin él no tendríamos de dónde sacar ese testigo. La demostración del teorema es por diagonalización (se fabrica una máquina que, en espacio n^2 , contradice a toda máquina de espacio lineal), pero para este ejercicio lo usamos como caja negra.

Lema 2. $\text{ESPACIO}(n)$ *no* es cerrada para reducciones en espacio logarítmico.

▷ **Concepto:** La técnica del “padding” (relleno)

Padding = alargar artificialmente la entrada añadiendo símbolos de relleno. ¿Para qué sirve? El coste en espacio se mide *respecto a la longitud de la entrada*. Si una entrada x requiere espacio $|x|^2$ pero la “estiramos” a longitud $N = |x|^2$, ese mismo coste $|x|^2 = N$ pasa a ser *lineal en N* . El relleno “diluye” la dificultad: el problema difícil se disfraza de fácil al medirlo sobre una entrada mayor. Es el truco que provoca el fallo de cierre.

Demostración. Por la jerarquía, $\text{ESPACIO}(n) \subsetneq \text{ESPACIO}(n^2)$; fijamos un lenguaje testigo

$$A \in \text{ESPACIO}(n^2) \setminus \text{ESPACIO}(n)$$

(existe por ser la inclusión estricta: decidible en espacio cuadrático, pero *no* en lineal).

Definimos su versión rellenada. Añadimos un símbolo nuevo $\# \notin \Sigma$:

$$B = \{ x\#^t : x \in A, t = |x|^2 - |x| \},$$

es decir, a cada $x \in A$ le pegamos relleno hasta que la longitud total sea $N = |x|^2$.

(i) $B \in \text{ESPACIO}(n)$. Dada z de longitud N : comprobamos que tiene forma $x\#^t$ con $t = |x|^2 - |x|$ (contando, en $O(\log N)$), extraemos x y ejecutamos el decididor de A , que usa espacio $O(|x|^2)$. Pero $|x|^2 = N$, así que es $O(N)$, *lineal en la entrada z* . Luego $B \in \text{ESPACIO}(N) = \text{ESPACIO}(n)$.

(ii) $A \propto B$ **en espacio log**. La reducción $f(x) = x\#^{|x|^2 - |x|}$ se calcula en $O(\log |x|)$: copiar x y escribir $|x|^2 - |x|$ símbolos $\#$ llevando solo un contador. Y $x \in A \iff f(x) \in B$.

(iii) **Conclusión.** Tenemos $A \propto B$ con $B \in \text{ESPACIO}(n)$. Si $\text{ESPACIO}(n)$ fuera cerrada para log-reducciones, debería ser $A \in \text{ESPACIO}(n)$. Pero $A \notin \text{ESPACIO}(n)$ por construcción. *Contradicción.* Luego $\text{ESPACIO}(n)$ **no** es cerrada. ■

¿Por qué este paso?

Reúne las dos piezas: B es “ A disfrazado de fácil” por el relleno, y la reducción f es solo “poner el disfraz”, que es trivial (log espacio). Si la clase respetara las reducciones, el disfraz

haría entrar a A en $\text{ESPACIO}(n)$ — pero sabemos que A es genuinamente difícil. La única salida lógica es que $\text{ESPACIO}(n)$ *no* respeta las reducciones.

Conclusión: $\text{NP} \neq \text{ESPACIO}(n)$

Demostración. Tenemos:

- NP *sí* es cerrada para log-reducciones (Paso 1);
- $\text{ESPACIO}(n)$ *no* lo es (Paso 2).

El cierre para log-reducciones es una propiedad que una clase tiene o no tiene; dos clases iguales coincidirían en ella. Como NP la tiene y $\text{ESPACIO}(n)$ no, forzosamente

$$\text{NP} \neq \text{ESPACIO}(n).$$

■

Nota (recapitulación del esquema, por si cae en examen). El patrón completo es: (1) elige una propiedad “ P es cerrada para cierto tipo de reducción”; (2) demuestra que una clase la cumple (aquí, vía certificado + “log-espacio $\Rightarrow$ tiempo polinómico”); (3) demuestra que la otra no, *exhibiendo* un contraejemplo $A \propto B$ con B dentro y A fuera, donde la separación $A \notin \text{clase}$ viene de un teorema de jerarquía y la reducción es un *padding* trivial; (4) concluye que las clases difieren. Los únicos ingredientes “pesados” son el teorema de jerarquía (diagonalización) y la idea de padding; el resto es encadenar definiciones.

Fin de la Relación 4 (versión comentada).

Documento de estudio. Resultados estándar en teoría de la complejidad (Sipser; Papadimitriou; Garey–Johnson); las demostraciones siguen los esquemas habituales de esos textos y la terminología de la asignatura.